

Task 18

Car Dataset

Name: Sarthak Bharat Sapkal
College Name: Dr. Panjabrao Deshmukh Polytechnic, Amravati
Branch: Computer Engineering
Domain: AIML

Ford Car Dataset: Data Reading, Loading, Exploring and Understanding

Introduction

Dataset: A dataset is a structured collection of related information organized in the form of rows and columns. Each row represents an individual record or observation, while each column represents a specific feature or attribute of that record. Datasets are used for analysis, visualization, and building machine learning models.

Data Reading: Data reading is the process of importing data from external files such as CSV, Excel, or text files into a Python environment. Libraries like Pandas are commonly used to read and access datasets efficiently.

Data Loading: Data loading refers to storing the imported dataset into the computer's memory so that it can be processed, cleaned, and analyzed. After loading, the data can be accessed using Python commands for further operations.

Data Exploring: Data exploration is the process of examining the dataset to understand its structure and characteristics. It includes checking:

- Number of rows and columns
- Column names
- Data types
- Missing values
- Duplicate records
- Statistical information

Data Understanding: Data understanding involves analyzing the meaning and importance of each feature in the dataset. It helps identify:

- What each column represents
- Relationships between variables
- Patterns and trends in the data
- Data quality issues such as missing values or outliers

For the Ford Car Dataset, understanding the features helps determine how factors like year, mileage, engine size, fuel type, and transmission influence the car's selling price.

Libraries Imported

1) import pandas as pd

Pandas (pd) is a Python library used for data manipulation and analysis. It provides data structures such as DataFrames that allow users to:

- Read datasets from files (CSV, Excel, etc.)
- View and modify data
- Handle missing values
- Remove duplicate records
- Perform statistical analysis

2) import matplotlib.pyplot as plt

Matplotlib (plt) is a Python library used for creating data visualizations and graphical representations. It helps to create:

- Histograms
- Bar charts
- Line graphs
- Scatter plots

These visualizations help understand data patterns and trends.

3) import seaborn as sns

Seaborn (sns) is a Python visualization library built on top of Matplotlib. It is mainly used for creating attractive and informative statistical graphs. It helps in creating:

- Count plots
- Heatmaps
- Distribution plots
- Correlation visualizations

The dataset contains both:

- **Numerical features:** price, year, mileage, tax, mpg, engineSize
- **Categorical features:** model, transmission, fuelType

Que 1 : Data Loading & Initial Analysis

Importing Libraries:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

File Uploading:

```
from google.colab import files
uploaded=files.upload()

Choose Files | ford.csv
ford.csv(text/csv) - 910937 bytes, last modified: 14/7/2026 - 100% done
Saving ford.csv to ford.csv
```

```
df.head(10)
```

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
0	Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
1	Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
2	Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
3	Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
4	Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0
5	Fiesta	2015	10500	Manual	35432	Petrol	145	47.9	1.6
6	Puma	2019	22500	Manual	2029	Petrol	145	50.4	1.0
7	Fiesta	2017	9000	Manual	13054	Petrol	145	54.3	1.2
8	Kuga	2019	25500	Automatic	6894	Diesel	145	42.2	2.0
9	Focus	2018	10000	Manual	48141	Petrol	145	61.4	1.0

df.head(10) — first 10 records of the dataset

```
df.tail()
```

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
17961	B-MAX	2017	8999	Manual	16700	Petrol	150	47.1	1.4
17962	B-MAX	2014	7499	Manual	40700	Petrol	30	57.7	1.0
17963	Focus	2015	9999	Manual	7010	Diesel	20	67.3	1.6
17964	KA	2018	8299	Manual	5007	Petrol	145	57.7	1.2
17965	Focus	2015	8299	Manual	5007	Petrol	22	57.7	1.0

df.tail() — last records of the dataset

```
df.shape

(17966, 9)
```

df.shape — dataset dimensions

df.dtypes	
	0
model	object
year	int64
price	int64
transmission	object
mileage	int64
fuelType	object
tax	int64
mpg	float64
engineSize	float64
dtype: object	

df.dtypes — column data types

Observations

- The dataset contains both numerical (price, mileage, year, engineSize, mpg) and categorical (model, transmission, fuelType) features.
- The target variable is *price*, because it is the value that needs to be predicted using the remaining features.
- Numerical features such as mileage, year, engineSize, and mpg can be used for statistical analysis, visualization, and correlation with the target variable.

Que 2 : Missing & Duplicate Values

```
df.isnull().sum()
```

```
0
model    0
year     0
price    0
transmission  0
mileage   0
fuelType  0
tax       0
mpg       0
engineSize  0
dtype: int64
```

df.isnull().sum() — checking for missing values

```
df.duplicated().sum()
```

```
np.int64(154)
```

df.duplicated().sum() — checking for duplicate rows

```
df.drop_duplicates()
```

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
0	Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
1	Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
2	Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
3	Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
4	Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0
...	...	...	...	...	...	...	...	...	...
17961	B-MAX	2017	8999	Manual	16700	Petrol	150	47.1	1.4
17962	B-MAX	2014	7499	Manual	40700	Petrol	30	57.7	1.0
17963	Focus	2015	9999	Manual	7010	Diesel	20	67.3	1.6
17964	KA	2018	8299	Manual	5007	Petrol	145	57.7	1.2
17965	Focus	2015	8299	Manual	5007	Petrol	22	57.7	1.0

17812 rows × 9 columns

df.drop_duplicates() — removing duplicate rows

Summary

- The dataset was checked for missing values and duplicate rows.
- No missing values were found in any column.
- Duplicate rows were identified and removed using `drop_duplicates()`.
- The cleaned dataset is now ready for further analysis.

Que 3 : Statistical Summary

```
df.describe()
print("Minimum Price:",df["price"].min())
print("Maximum Price:",df["price"].max())
print("Mean Price:",df["price"].mean())
print("Median Price:",df["price"].median())
```

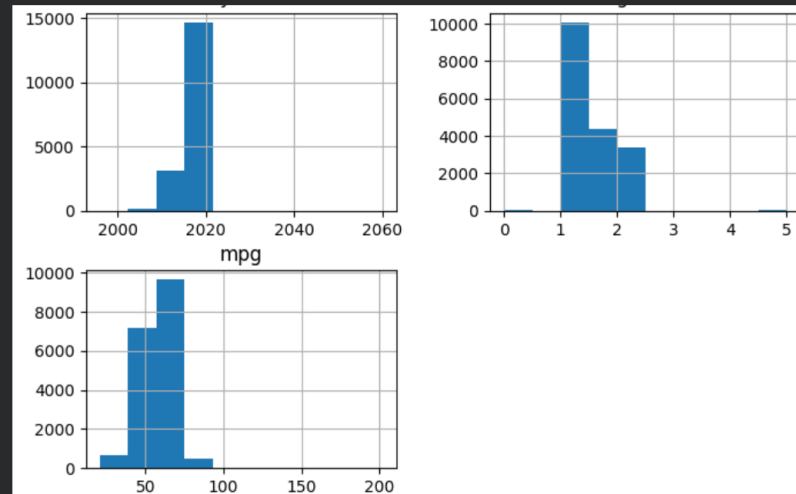
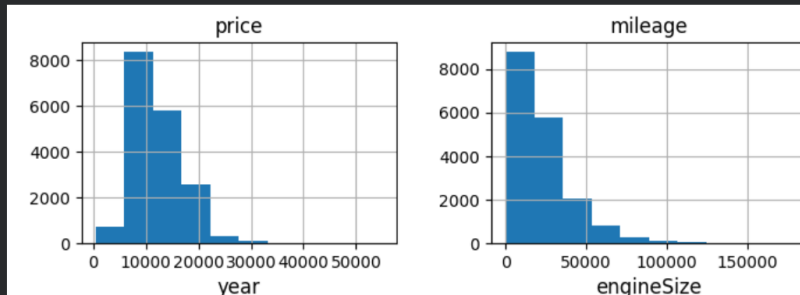
```
Minimum Price: 495
Maximum Price: 54995
Mean Price: 12279.534843593454
Median Price: 11291.0
```

df.describe() and price statistics (min, max, mean, median)

Que 4 : Histogram of Numeric Features

import matplotlib.pyplot as plt

```
import matplotlib.pyplot as plt
df[["price", "mileage", "year", "engineSize", "mpg"]].hist(figsize=(8,8))
plt.show()
```



Histograms of price, mileage, year, engineSize, and mpg

Observations

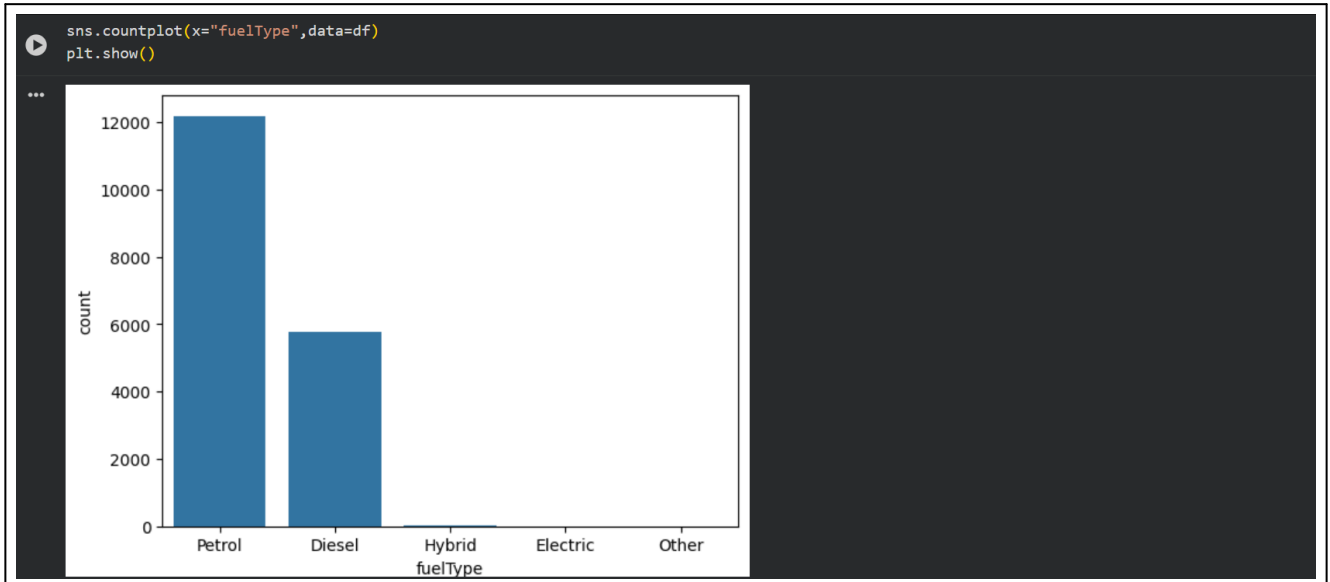
- **Price:** The distribution of car prices is uneven, with most vehicles falling in the lower price range and only a few expensive cars.
- **Mileage:** Most cars have moderate mileage, while vehicles with extremely high mileage are less common.

- **Year:** The dataset contains more recently manufactured cars than older ones, indicating that newer vehicles are more frequently listed.
- **Engine Size:** Smaller engine sizes are more common, whereas larger engine sizes appear less frequently in the dataset.
- **MPG:** Most vehicles have average fuel efficiency values, with only a small number having very low or very high MPG.

Que 5 : Count Plots of Categorical Features

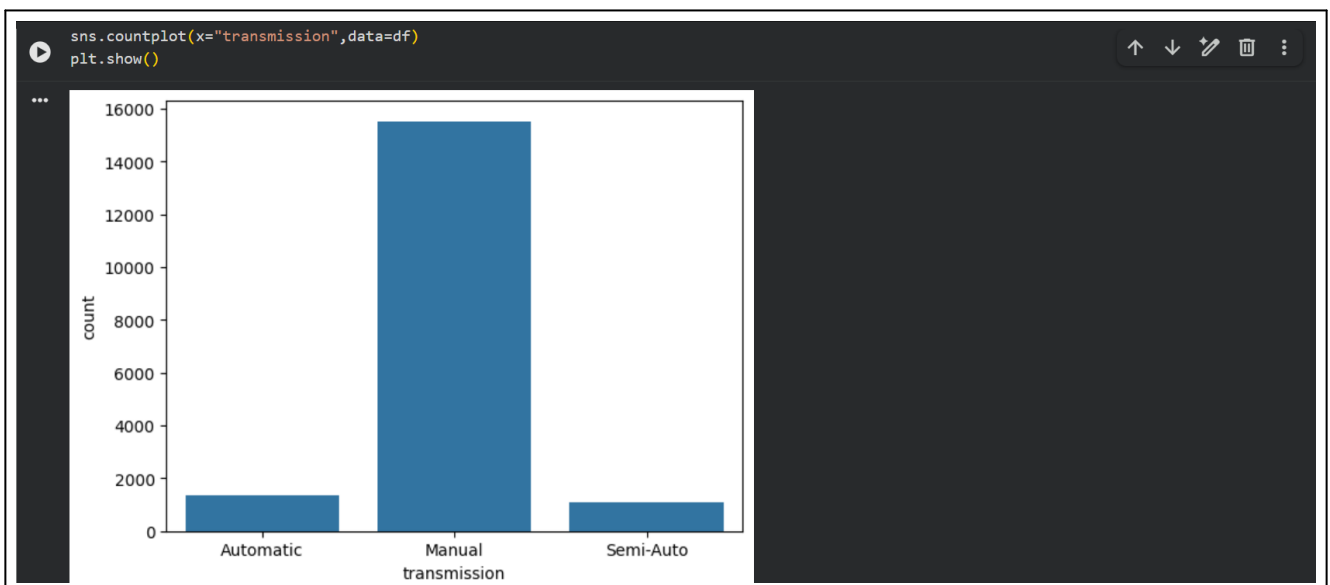
```
import seaborn as sns
```

FuelType



Count plot of fuelType

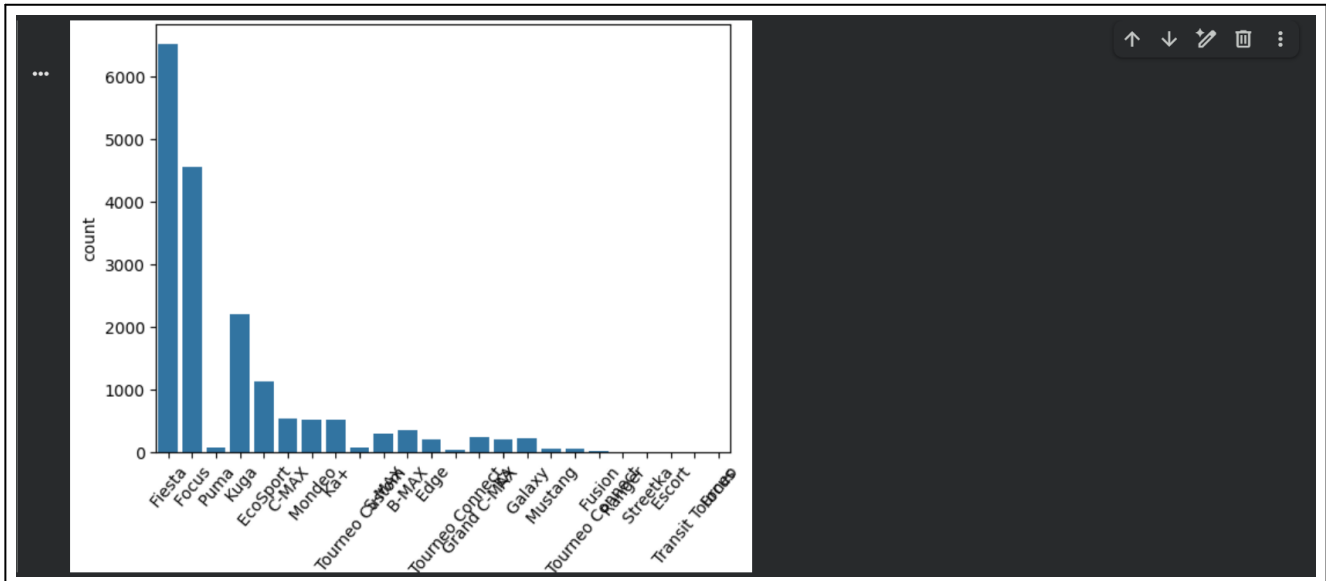
Transmission



Count plot of transmission

Model

```
sns.countplot(x="model",data=df)  
plt.xticks(rotation=50)  
plt.show()
```



Count plot of model

Observations

- **Fuel Type:** Petrol and Diesel vehicles are the most common in the dataset, while Hybrid and Electric vehicles appear less frequently.
- **Transmission:** Manual transmission cars are more common than Automatic cars, indicating higher demand or availability of manual vehicles.
- **Model:** Some Ford models appear much more frequently than others, showing that a few popular models dominate the market.

Market Trends

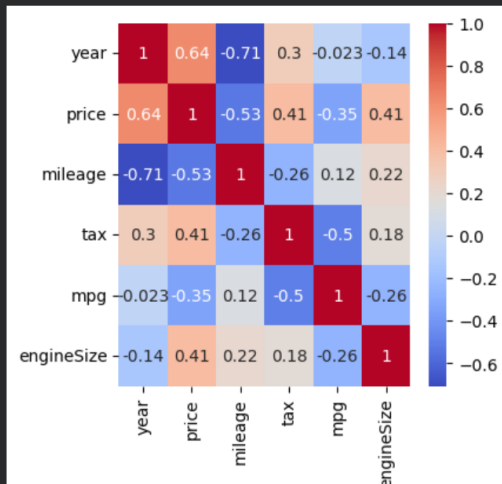
- Petrol and Diesel cars dominate the used car market.
- Manual transmission vehicles have a larger market share, suggesting they are more commonly available.
- A small number of Ford models account for most of the listings, indicating higher customer demand and popularity.
- Hybrid and Electric vehicles have lower representation, showing that they are less common in this dataset.

Summary

- The count plots show the frequency of different categories in the dataset.
- Petrol/Diesel fuel types and Manual transmission are the most common.
- A few Ford models dominate the dataset, while other models appear less often.
- These plots provide insights into customer preferences and the availability of different vehicle types in the market.

Q6 : Correlation Heatmap

```
plt.figure(figsize=(4,4))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")
plt.show()
```



Correlation heatmap of numeric features

Summary

- The correlation heatmap helps identify relationships between numeric variables.
- Year shows the strongest positive relationship with Price, while Mileage shows a negative relationship with Price.
- Engine Size also contributes positively to price.
- These features are useful for predicting car prices in a machine learning model.

Que 7 : Feature Identification

```
from matplotlib import axis
x = df.drop("price",axis=1)
y = df["price"]

print(x.columns)
print(y.name)

*** Index(['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg',
        'engineSize'],
        dtype='object')
price
```

Splitting the dataset into input (x) and output (y) variables

Que 8 : Encoding Categorical Variables

```
encoded_df = pd.get_dummies(df)
print(encoded_df.head())

***
```

	year	price	mileage	tax	mpg	engineSize	model_ B-MAX	model_ C-MAX	\
0	2017	12000	15944	150	57.7	1.0	False	False	
1	2018	14000	9083	150	57.7	1.0	False	False	
2	2017	13000	12456	150	57.7	1.0	False	False	
3	2019	17500	10460	145	40.3	1.5	False	False	
4	2019	16500	1482	145	48.7	1.0	False	False	

	model_ EcoSport	model_ Edge	...	model_ Transit	Tourneo	model_Focus	\
0	False	False	...	False	False	False	
1	False	False	...	False	False	False	
2	False	False	...	False	False	False	
3	False	False	...	False	False	False	
4	False	False	...	False	False	False	

	transmission_Automatic	transmission_Manual	transmission_Semi-Auto	\
0	True	False	False	
1	False	True	False	
2	False	True	False	
3	False	True	False	
4	True	False	False	

```
transmission_Automatic transmission_Manual transmission_Semi-Auto \
0 True False False False
*** 1 False True False False
2 False True False False
3 False True False False
4 True False False False

fuelType_Diesel fuelType_Electric fuelType_Hybrid fuelType_Other \
0 False False False False
1 False False False False
2 False False False False
3 False False False False
4 False False False False

fuelType_Petrol
0 True
1 True
2 True
3 True
4 True

[5 rows x 38 columns]
```

pd.get_dummies() — one-hot encoding of categorical columns

Que 9 : Feature Scaling

```
from sklearn.preprocessing import StandardScaler
x = encoded_df.drop("price", axis=1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(x)
print(X_scaled[:5])
```

```
[[ 0.06512772 -0.38099808  0.59135805 -0.02044162 -0.81138621 -0.14197836
 -0.1765381  -0.2606582  -0.10822676 -0.00746082  1.31908019 -0.58562028
 ...
 -0.02985573 -0.11337443 -0.11806709 -0.05740036 -0.10583261 -0.17451642
 -0.37596624 -0.17366793 -0.05641593 -0.06687879 -0.00746082 -0.1294278
 -0.01055148 -0.04289735 -0.06209183 -0.00746082 -0.00746082  3.49293398
 -2.51774746 -0.25377066 -0.68712465 -0.01055148 -0.0350148  -0.00746082
  0.68932007]
 [ 0.55286624 -0.73335899  0.59135805 -0.02044162 -0.81138621 -0.14197836
 -0.1765381  -0.2606582  -0.10822676 -0.00746082 -0.75810402  1.70759113
 -0.02985573 -0.11337443 -0.11806709 -0.05740036 -0.10583261 -0.17451642
 -0.37596624 -0.17366793 -0.05641593 -0.06687879 -0.00746082 -0.1294278
 -0.01055148 -0.04289735 -0.06209183 -0.00746082 -0.00746082 -0.28629227
 -0.39718042 -0.25377066 -0.68712465 -0.01055148 -0.0350148  -0.00746082
  0.68932007]
 [ 0.06512772 -0.56013157  0.59135805 -0.02044162 -0.81138621 -0.14197836
 -0.1765381  -0.2606582  -0.10822676 -0.00746082 -0.75810402  1.70759113
 -0.02985573 -0.11337443 -0.11806709 -0.05740036 -0.10583261 -0.17451642
 -0.37596624 -0.17366793 -0.05641593 -0.06687879 -0.00746082 -0.1294278
 -0.01055148 -0.04289735 -0.06209183 -0.00746082 -0.00746082 -0.28629227
 -0.39718042 -0.25377066 -0.68712465 -0.01055148 -0.0350148  -0.00746082
  0.68932007]
 [ 1.04060476 -0.6626403  0.51072684 -1.73888989  0.34507023 -0.14197836
 -0.1765381  -0.2606582  -0.10822676 -0.00746082  1.31908019 -0.58562028
 -0.02985573 -0.11337443 -0.11806709 -0.05740036 -0.10583261 -0.17451642
 -0.37596624 -0.17366793 -0.05641593 -0.06687879 -0.00746082 -0.1294278
 -0.01055148 -0.04289735 -0.06209183 -0.00746082 -0.00746082 -0.28629227
 -2.51774746 -0.25377066 -0.68712465 -0.01055148 -0.0350148  -0.00746082
  0.68932007]
```

StandardScaler — scaling numeric features

Que 10 : Mini Project — Complete Preprocessing Pipeline

```
import pandas as pd
```

1. Load and clean the data (handle missing & duplicate values)

```
0.68932007]
[ 1.04060476 -1.12372415  0.51072684 -0.90929417 -0.81138621 -0.14197836
 -0.1765381  -0.2606582  -0.10822676 -0.00746082  1.31908019 -0.58562028
 ...
 -0.02985573 -0.11337443 -0.11806709 -0.05740036 -0.10583261 -0.17451642
 -0.37596624 -0.17366793 -0.05641593 -0.06687879 -0.00746082 -0.1294278
 -0.01055148 -0.04289735 -0.06209183 -0.00746082 -0.00746082  3.49293398
 -2.51774746 -0.25377066 -0.68712465 -0.01055148 -0.0350148  -0.00746082
  0.68932007]
```

```
df=pd.read_csv("ford.csv")
df.head()
```

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
0	Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
1	Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
2	Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
3	Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
4	Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0

```
▶ print("Missing Values")
  print(df.isnull().sum())

*** Missing Values
model      0
year       0
price      0
transmission 0
mileage    0
fuelType   0
tax        0
mpg        0
engineSize 0
dtype: int64
```

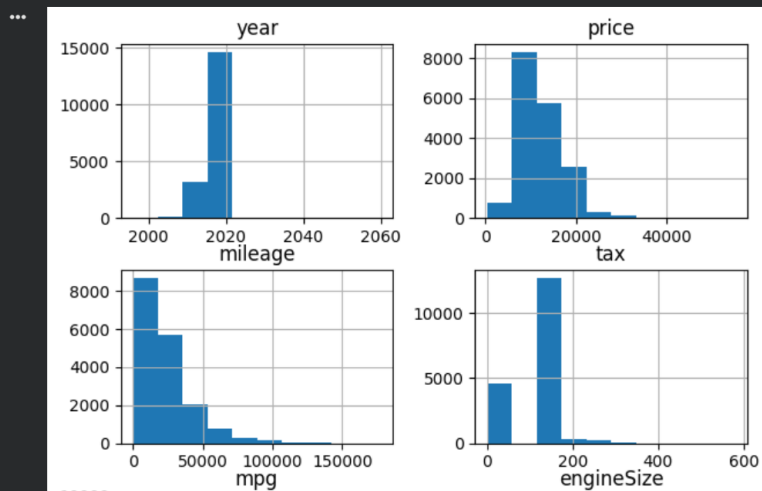
Loading the dataset and checking for missing values

2. Perform EDA (histograms + count plots + heatmap)

```
df=df.drop_duplicates()
```

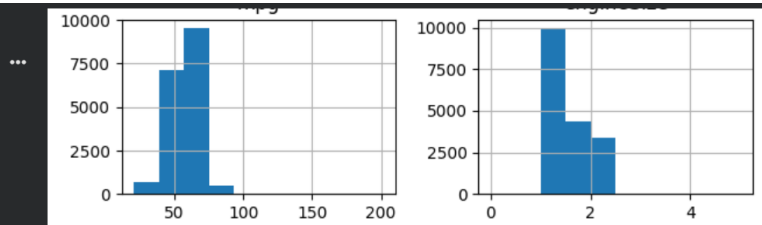
```
df=df.fillna(df.median(numeric_only=True))  
df=df.fillna(df.mode().iloc[0])
```

```
df.hist(figsize=(7,7))  
plt.show()
```

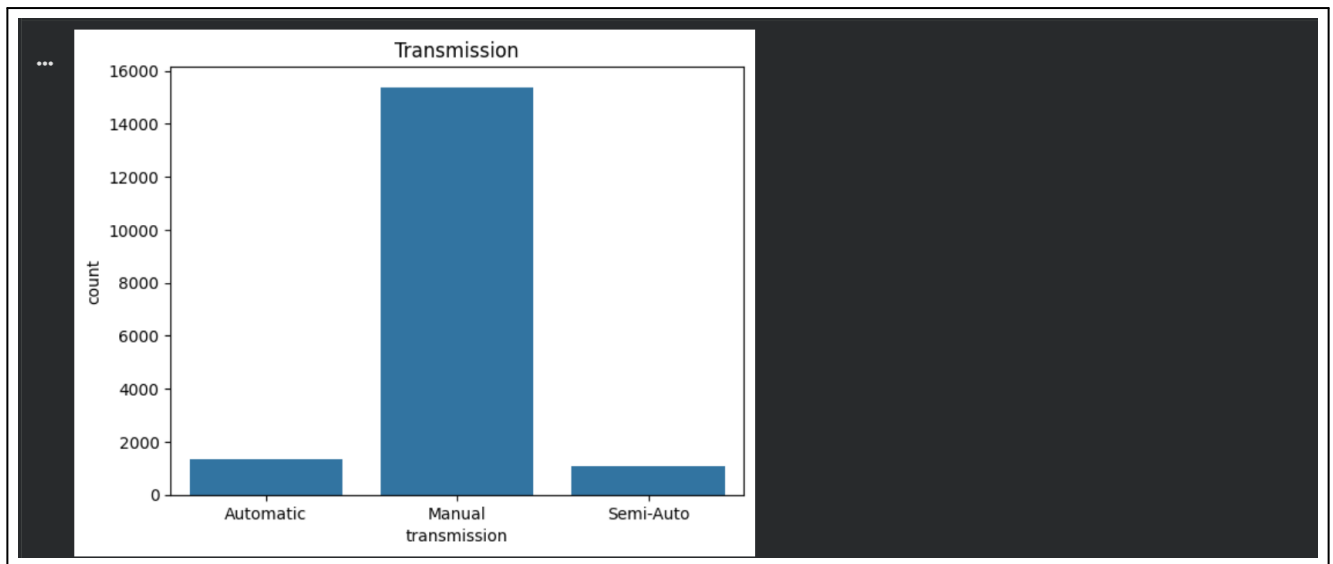


Histograms of numeric features

Transmission



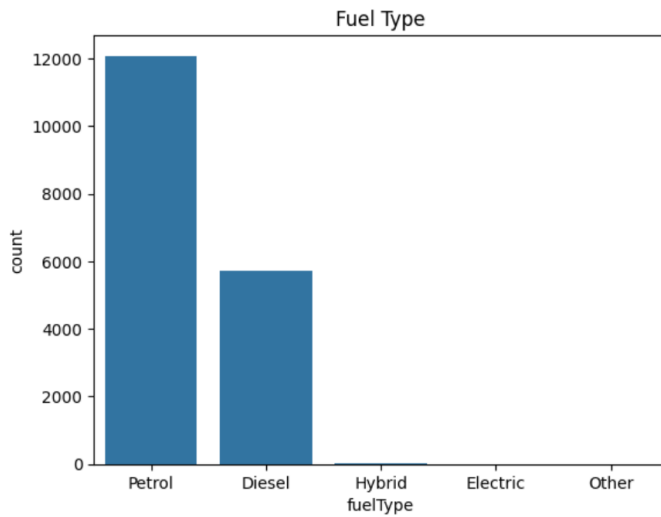
```
sns.countplot(x="transmission",data=df)  
plt.title("Transmission")  
plt.show()
```



Count plot of transmission

FuelType

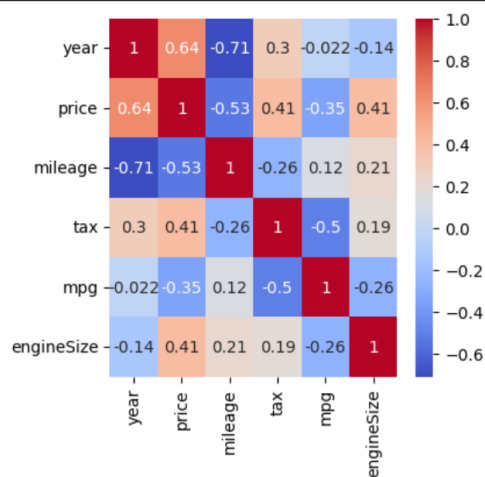
```
sns.countplot(x="fuelType",data=df)
plt.title("Fuel Type")
plt.show()
```



Count plot of fuelType

Heatmap

```
plt.figure(figsize=(4,4))
sns.heatmap(df.corr(numeric_only=True),annot=True,cmap="coolwarm")
plt.show()
```



Correlation heatmap of numeric features

3. Identify input and output features

```
x = df.drop("price",axis=1)
y = df["price"]

print("\n Input Variable:")
print(x.columns)
print("\n Output Variable:")
print(y.name)

Input Variable:
Index(['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg',
      'engineSize'],
      dtype='object')

Output Variable:
price
```

Splitting the dataset into input and output variables

4. Encode categorical variables

```
x=pd.get_dummies(x,drop_first=True)
print("\n Encoded Data:")
print(x.head())
```

```
Encoded Data:
   year  mileage  tax  mpg  engineSize  model_ C-MAX  model_ EcoSport \
0  2017   15944  150  57.7         1.0        False        False
1  2018    9083  150  57.7         1.0        False        False
2  2017   12456  150  57.7         1.0        False        False
3  2019   10460  145  40.3         1.5        False        False
4  2019    1482  145  48.7         1.0        False        False

   model_ Edge  model_ Escort  model_ Fiesta  ...  model_ Tourneo Connect \
0        False        False         True  ...                False
1        False        False         False  ...                False
2        False        False         False  ...                False
3        False        False         True   ...                False
4        False        False         True   ...                False

   model_ Tourneo Custom  model_ Transit Tourneo  model_Focus \
0                False                False        False
1                False                False        False
2                False                False        False
3                False                False        False
4                False                False        False
```

pd.get_dummies() — encoding categorical columns

5. Scale the numeric features

```
... transmission_Manual  transmission_Semi-Auto  fuelType_Electric \
0                False                False                False
1                True                False                False
2                True                False                False
3                True                False                False
4                False                False                False

   fuelType_Hybrid  fuelType_Other  fuelType_Petrol
0                False                False                True
1                False                False                True
2                False                False                True
3                False                False                True
4                False                False                True

[5 rows x 34 columns]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(x)
print("\n Preprocessing completed Successfully:")
```

```
Preprocessing completed Successfully:
```

StandardScaler — preprocessing pipeline completed